

Inheritance

Unit 2

Contents

- Basic inheritance
- Extending built-ins
- Overriding and super
- Multiple inheritance
- Different sets of arguments
- Polymorphism
- Abstract base classes

Basic inheritance

Inheritance is an important aspect of the object-oriented paradigm. Inheritance provides code reusability to the program because we can use an existing class to create a new class instead of creating it from scratch.

Syntax:

```
class super_class: # define a superclass
```

```
    # attributes and method definition
```

```
class sub_class(super_class): # inheritance
```

```
    # attributes and method of super_class
```

```
    # attributes and method of sub_class
```

Inheritance is an is-a relationship. That is, we use inheritance only if there exists an is-a relationship between two classes.

Constructors in Inheritance

Constructor is special methods that is used to initialize the instance variable of a class. In the constructor, we create the instance variables them with same starting values.

E.g. class father:

```
def __init__(self):  
    self.property=800000  
  
def display_property(self):  
    print("Fathers property=",self.property)
```

```
Class son(father):  
    Pass  
s=son()  
s.display_property()
```

Overriding and super

Super () is built-in method which is useful to call the super class constructor or method from the sub class.

```
class <base_class>:
    def __init__(self, parameters...):
        <function body>
    def <function name>(self):
        <function body>
class <Derived_class>(<base_class>):
    def __init__(self, parameters...):
        super().__init__(<parameters...>)
        <function body>
    def display_property(self):
        <function body>
```

Types of Inheritance

The main advantage of inheritance is code reusability. The member of super class are reusable in the sub class.

In python all classes are built from a single super class called “**object**”.

class MyClass(object):

is equal to

class MyClass:

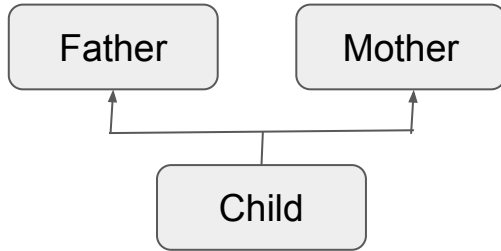
There are two types in Inheritance: 1) Single inheritance 2) Multiple inheritance

Single Inheritance

Deriving one or more sub classes from a single base class is called single inheritance.

Multiple inheritance

Deriving sub classes from multiple (or more than one base class) is called multiple inheritance.



```
class father:
    def height(self):
        print(" Height is 6.0 foot")
class mother:
    def color(self):
        print("color is brown")
class child:
    pass
c=child()
print(" Child inherited qualities:")
c.height()
c.color()
```

Problem in Multiple Inheritance

If the sub class has a constructor, it overrides the super class constructor and hence the super class constructor is not available to the subclass. But writing constructor is very common to initialize the instance variable.

```
class A(object):  
    def __init__(self):  
        self.a= "A"  
        print(self.a)  
class B(object):  
    def __init__(self):  
        self.b= "B"  
        print(self.b)  
class C(A,B):  
    def __init__(self):  
        self.c= "C"  
        print(self.c)
```

```
obj=C()
```

Overriding Super class constructor and method

When the programmer writes a constructor in the sub class, the super class constructor is not available to the sub class. In this case, only the sub class constructor is accessible from the sub class object. That means the sub class constructor is replacing the super class constructor. This is called *constructor overloading*. Similar for method, This is called *method overriding*.

Polymorphism

If something exhibits various forms, it is called polymorphism.

A variable may store different types of data , an object may exhibit different behaviours in different context or a method may perform different tasks in python.

This type of behaviour is called polymorphism.

- Operator overloading
- Method Overloading
- Method Overriding

```
class A:
    def talk(self):
        print(" I am A")
class B:
    def talk(self):
        print("I am B")
def call_talk(obj):
    obj.talk()
```

```
x=A()
call_talk(x)
x=B()
call_talk(x)
```

Operator overloading

Operator is symbol that performs some action. (e.g. + perform addition)

When an operator performs different operations, it is said to exhibit polymorphism.

- Overloading + operator
- Use `def __add__(self,parameter):` // This methods are called magic method

Operators and corresponding magic methods

Operator	Magic Method	Operator	Magic Method
+	object.__add__(self, other)	-	object.__sub__(self, other)
*	object.__mul__(self, other)	/	object.__div__(self, other)
**	object.__pow__(self, other)	%	object.__mod__(self, other)
==	object.__eq__(self, other)	>	object.__gt__(self, other)

Method Overloading

If a method is written such that it can perform more than one task, it is called method overloading.

e.g. class myclass:

```
def sum(self,a=None,b=None,c=None):  
    if a!=None and b!=None and c!=None:  
        print ("Sum of THREE number is", a+b+c)  
    elif a=None,b=None:  
        print ("Sum of TWO number is", a+b)  
    else:  
        print("Enter two or three arguments")
```

Abstract base classes

An abstract class can be considered a blueprint for other classes. It allows you to create a set of methods that must be created within any child classes built from the abstract class.

A class that contains one or more abstract methods is called an abstract class. An abstract method is a method that has a declaration but does not have an implementation.

Working on Python Abstract classes

By default, Python does not provide abstract classes. Python comes with a module that provides the base for defining Abstract Base classes(ABC) and that module name is ABC.

ABC works by decorating methods of the base class as an abstract and then registering concrete classes as implementations of the abstract base.

A method becomes abstract when decorated with the keyword `@abstractmethod`.

e.g

```
from abc import ABC,
abstractmethod
class Polygon(ABC):
    @abstractmethod
    def noofsides(self):
        pass
class Triangle(Polygon):
    def noofsides(self):
        print("I have 3
sides")
class Pentagon(Polygon):
    def noofsides(self):
        print("I have 5
sides")
class Hexagon(Polygon):
    def noofsides(self):
        print("I have 6
sides")
class Quadrilateral(Polygon):
    def noofsides(self):
        print("I have 4
sides")

R = Triangle()
R.noofsides()
K = Quadrilateral()
K.noofsides()
R = Pentagon()
R.noofsides()
K = Hexagon()
K.noofsides()
```